

Complete Step-by-Step Manual for Lead Enquiry Audit History Tracking

This guide is for your existing Lead Enquiry module files:

- FormLeadEnquiry.js
- ExcelBulkUpload.js
- ViewLeadEnquiry.js
- TabLeadEnquiry.js

We will implement:

Feature	Status
Track Create	✓
Track Update	✓
Track Delete	✓
Track User	✓
Track Time	✓
Track Old/New Data	✓
Auto Table Creation	✓

STEP 1 — CREATE HISTORY MODEL FILE

Go to:

backend/models

Create new file:

LeadEnquiryHistory.js

STEP 2 — PASTE THIS MODEL CODE

Inside:

models/LeadEnquiryHistory.js

paste:

```
import { DataTypes } from "sequelize";
import sequelize from "../config/db.js";

const LeadEnquiryHistory = sequelize.define(
  "lead_enquiry_history",
  {
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true,
    },

    leadId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },

    leadReferenceNo: {
      type: DataTypes.STRING,
    },

    actionType: {
      type: DataTypes.STRING,
      allowNull: false,
    },

    oldData: {
      type: DataTypes.JSON("long"),
```

```
},
```

```
newData: {  
  type: DataTypes.JSON("long"),  
},
```

```
changedById: {  
  type: DataTypes.STRING,  
},
```

```
changedByName: {  
  type: DataTypes.STRING,  
},
```

```
changedByRole: {  
  type: DataTypes.STRING,  
},
```

```
changedBySbu: {  
  type: DataTypes.STRING,  
},
```

```
},
```

```
{  
  freezeTableName: true,  
  timestamps: true,  
  createdAt: "actionDate",  
  updatedAt: false,  
}
```

```
);
```

```
export default LeadEnquiryHistory;
```

STEP 3 — IMPORT MODEL

Now go to your main backend file.

Possible files:

server.js

OR

index.js

OR wherever you imported models.

Add:

```
import './models/LeadEnquiryHistory.js';
```

STEP 4 — RESTART SERVER

Run:

npm start

OR

nodemon index.js

STEP 5 — VERIFY TABLE CREATED

Open MySQL.

You should now see:

lead_enquiry_history

created automatically.

STEP 6 — CREATE HISTORY LOGGER UTILITY

Go to:

backend/utils

Create file:

logLeadHistory.js

STEP 7 — PASTE LOGGER CODE

Paste:

```
import LeadEnquiryHistory from "../models/LeadEnquiryHistory.js";
```

```
export const logLeadHistory = async ({
```

```
  leadId,
```

```
  leadReferenceNo,
```

```
  actionType,
```

```
  oldData,
```

```
  newData,
```

```
  user,
```

```
}) => {
```

```
  try {
```

```
    await LeadEnquiryHistory.create({
```

```
      leadId,
```

```
      leadReferenceNo,
```

```
      actionType,
```

```
      oldData,
```

```
      newData,
```

```
      changedById: user?.operatorId,
```

```
    changedByName: user?.operatorName,  
    changedByRole: user?.operatorRole,  
    changedBySbu: user?.operatorSbu,  
  });  
} catch (error) {  
  console.log("History Log Error:", error.message);  
}  
};
```

STEP 8 — MODIFY CREATE API

Go to your Lead Enquiry controller.

Example:

controllers/LeadEnquiryController.js

STEP 9 — IMPORT LOGGER

At top:

```
import { logLeadHistory } from "../utils/logLeadHistory.js";
```

STEP 10 — FIND CREATE FUNCTION

Find:

createLeadEnquiry

OR similar.

You will have something like:

```
const createdLead = await LeadEnquiry.create(payload);
```

STEP 11 — ADD HISTORY AFTER CREATE

Immediately BELOW create statement add:

```
await logLeadHistory({
```

```
leadId: createdLead.id,  
leadReferenceNo: createdLead.leadReferenceNo,  
  
actionType: "CREATE",  
  
oldData: null,  
newData: createdLead,  
  
user: {  
  operatorId: req.body.operatorId,  
  operatorName: req.body.operatorName,  
  operatorRole: req.body.operatorRole,  
  operatorSbu: req.body.operatorSbu,  
},  
});
```

STEP 12 — MODIFY UPDATE API

Find update controller.

Probably:

```
updateLeadEnquiry
```

STEP 13 — BEFORE UPDATE GET OLD DATA

Add BEFORE update:

```
const existingLead = await LeadEnquiry.findOne({  
  where: { id: req.body.id },  
});
```

STEP 14 — UPDATE RECORD

Example:

```
await LeadEnquiry.update(req.body, {  
  where: { id: req.body.id },  
});
```

STEP 15 — GET UPDATED DATA

Add:

```
const updatedLead = await LeadEnquiry.findOne({  
  where: { id: req.body.id },  
});
```

STEP 16 — ADD UPDATE HISTORY

Add:

```
await logLeadHistory({  
  leadId: updatedLead.id,  
  leadReferenceNo: updatedLead.leadReferenceNo,  
  
  actionType: "UPDATE",  
  
  oldData: existingLead,  
  newData: updatedLead,  
  
  user: {  
    operatorId: req.body.operatorId,  
    operatorName: req.body.operatorName,  
    operatorRole: req.body.operatorRole,  
    operatorSbu: req.body.operatorSbu,  
  },  
});
```

```
});
```

STEP 17 — MODIFY DELETE API

Find delete controller.

STEP 18 — GET OLD DATA BEFORE DELETE

Add:

```
const existingLead = await LeadEnquiry.findOne({  
  where: { id: req.body.id },  
});
```

STEP 19 — SAVE DELETE HISTORY

BEFORE delete:

```
await logLeadHistory({  
  leadId: existingLead.id,  
  leadReferenceNo: existingLead.leadReferenceNo,  
  
  actionType: "DELETE",  
  
  oldData: existingLead,  
  newData: null,  
  
  user: {  
    operatorId: req.body.operatorId,  
    operatorName: req.body.operatorName,  
    operatorRole: req.body.operatorRole,  
    operatorSbu: req.body.operatorSbu,  
  },
```

```
});
```

STEP 20 — DELETE RECORD

```
await LeadEnquiry.destroy({  
  where: { id: req.body.id },  
});
```

STEP 21 — FRONTEND UPDATE CHANGES

Go to:

ViewLeadEnquiry.js

Find:

```
const updatePayload = {  
  id: editingRow.id,  
  ...editingRow,  
};
```

STEP 22 — REPLACE WITH

```
let user = JSON.parse(localStorage.getItem("user")) || {};
```

```
const updatePayload = {  
  id: editingRow.id,  
  ...editingRow,
```

```
    operatorId: user.id,  
    operatorName: user.username,  
    operatorRole: user.userRole,  
    operatorSbu: user.sbu,  
};
```

STEP 23 — MODIFY DELETE PAYLOAD

Find:

```
const deleteData = {  
  id: id,  
};
```

STEP 24 — REPLACE WITH

```
let user = JSON.parse(localStorage.getItem("user")) || {};
```

```
const deleteData = {  
  id: id,  
  
  operatorId: user.id,  
  operatorName: user.username,  
  operatorRole: user.userRole,  
  operatorSbu: user.sbu,  
};
```

STEP 25 — BULK UPLOAD HISTORY

Go to backend bulk upload API.

Your frontend already sends:

- OperatorId
- OperatorName
- OperatorRole
- OperatorSBU

correctly

STEP 26 — INSIDE BULK LOOP ADD HISTORY

Example:

```
for (const row of excelData) {  
  
    const createdLead = await LeadEnquiry.create(row);  
  
    await logLeadHistory({  
        leadId: createdLead.id,  
        leadReferenceNo: createdLead.leadReferenceNo,  
  
        actionType: "CREATE",  
  
        oldData: null,  
        newData: createdLead,  
  
        user: {  
            operatorId: row.OperatorId,  
            operatorName: row.OperatorName,  
            operatorRole: row.OperatorRole,  
            operatorSbu: row.OperatorSBU,  
        },  
    });  
}
```

STEP 27 — OPTIONAL HISTORY API

Create new API:

GET /lead-enquiry-history/:leadId

STEP 28 — HISTORY CONTROLLER

```
export const getLeadHistory = async (req, res) => {  
  try {  
  
    const history = await LeadEnquiryHistory.findAll({  
      where: {  
        leadId: req.params.leadId,  
      },  
      order: [["actionDate", "DESC"]],  
    });  
  
    return res.status(200).json({  
      success: true,  
      data: history,  
    });  
  
    } catch (error) {  
  
      return res.status(500).json({  
        success: false,  
        message: error.message,  
      });  
    }  
  };  
};
```

STEP 29 — ADD ROUTE

Go to router file.

Add:

```
router.get(
  "/lead-enquiry-history/:leadId",
  getLeadHistory
);
```

STEP 30 — FINAL TESTING

Now test:

TEST CREATE

Create new lead.

Check:

lead_enquiry_history

You should see:

actionType

CREATE

TEST UPDATE

Edit row.

You should see:

actionType

UPDATE

with:

- oldData
 - newData
-

TEST DELETE

Delete row.

You should see:

actionType

DELETE

FINAL RESULT

You now have enterprise-level audit tracking:

Feature	Result
Who created	✓
Who edited	✓
Who deleted	✓
Timestamp	✓
Old data	✓
New data	✓
Bulk upload tracking	✓
Automatic table creation	✓
Full audit trail	✓

This is the same architecture used in CRM/ERP enterprise systems.